

# Chapter 3

## Software Quality Factors

# Software Quality Factors

- **The need for comprehensive Software Quality Requirements**
- **Classification of requirements into Software Quality Factors**
- **Product Operation Factors**
- **Product Revision Factors**
- **Product Transition Factors**
- **Alternative models of software quality factors**
- **Who is interested in defining quality requirements?**
- **Software compliance with Quality Factors**

# The Requirements Document

- Requirement Documentation (Specification) is one of the **most important elements** for achieving software quality
- Need to explore what constitutes a good software requirements document.
- Some SQA Models suggest 11-15 factors categorized; some fewer; some more
- Want to become familiar with these quality factors, and
- Who is really interested in them.
- The need for comprehensive software quality requirements is pervasive in numerous case studies (see a few in this chapter).
- (Where do the quality factors go??)

# Need for Comprehensive Software Quality Requirements

- Need for improving poor requirements documents is widespread
- Frequently lack quality factors such as: usability, reusability, maintainability, ...
- Software industry groups the long list of related attributes into what we call *quality factors*. (*Sometimes non-functional requirements*)
- Natural to assume an unequal emphasis on all quality factors.
- Emphasis varies from project to project
  - Scalability; maintainability; reliability...
- Let's look at some of the categories...

# Extra Thoughts

- Seems like in Software Engineering we concentrate on capturing, designing, implementing, and deploying with emphasis on **functional requirements**.
- Little (not none!) emphasis on the **non-functional requirements** (quality factors).
- In the RUP, non-functional requirements are captured in the Software Requirements Specification (SRS); functional requirement usually captured in Use Case stories.

# Non-functional requirements

- Usability requirement
- Serviceability requirement
- Manageability requirement
- Recoverability requirement
- Security requirement
- Data Integrity requirement
- Capacity requirement
- Availability requirement
- Scalability requirement
- Interoperability requirement
- Reliability requirement
- Maintainability requirement
- Regulatory requirement
- Environmental requirement

1. Users must change the initially assigned login password immediately after the first successful login. Moreover, the initial should never be reused.

2. Employees never allowed to update their salary information. Such attempt should be reported to the security administrator.

3. Every unsuccessful attempt by a user to access an item of data shall be recorded on an audit trail.

4. A website should be capable enough to handle 20 million users with affecting its performance

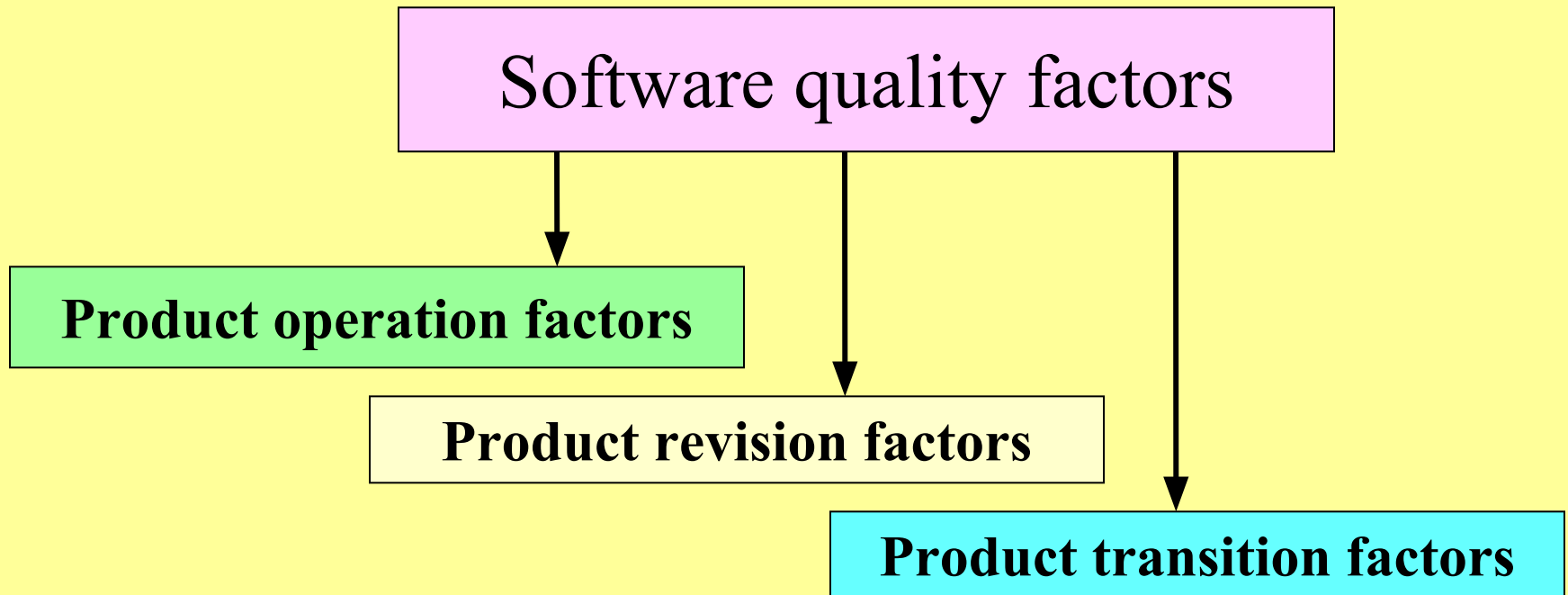
5. The software should be portable. So moving from one OS to other OS does not create any problem.

6. Privacy of information, the export of restricted technologies, intellectual property rights, etc. should be audited.

# McCall's Quality Factors

- McCall has 11 factors; Groups them into categories.
  - 1977; others have added, but this still prevail.
- Three categories:
  - Product Operation Factors
    - How well it runs....
    - Correctness, reliability, efficiency, integrity, and usability
  - Product Revision Factors
    - How well it can be changed, tested, and redeployed.
    - Maintainability; flexibility; testability
  - Product Transition Factors
    - How well it can be moved to different platforms and interface with other systems
    - Portability; Reusability; Interoperability
- Since these underpin the notion of quality factors and others who have added, reword or add one or two, we will spend time on these factors.

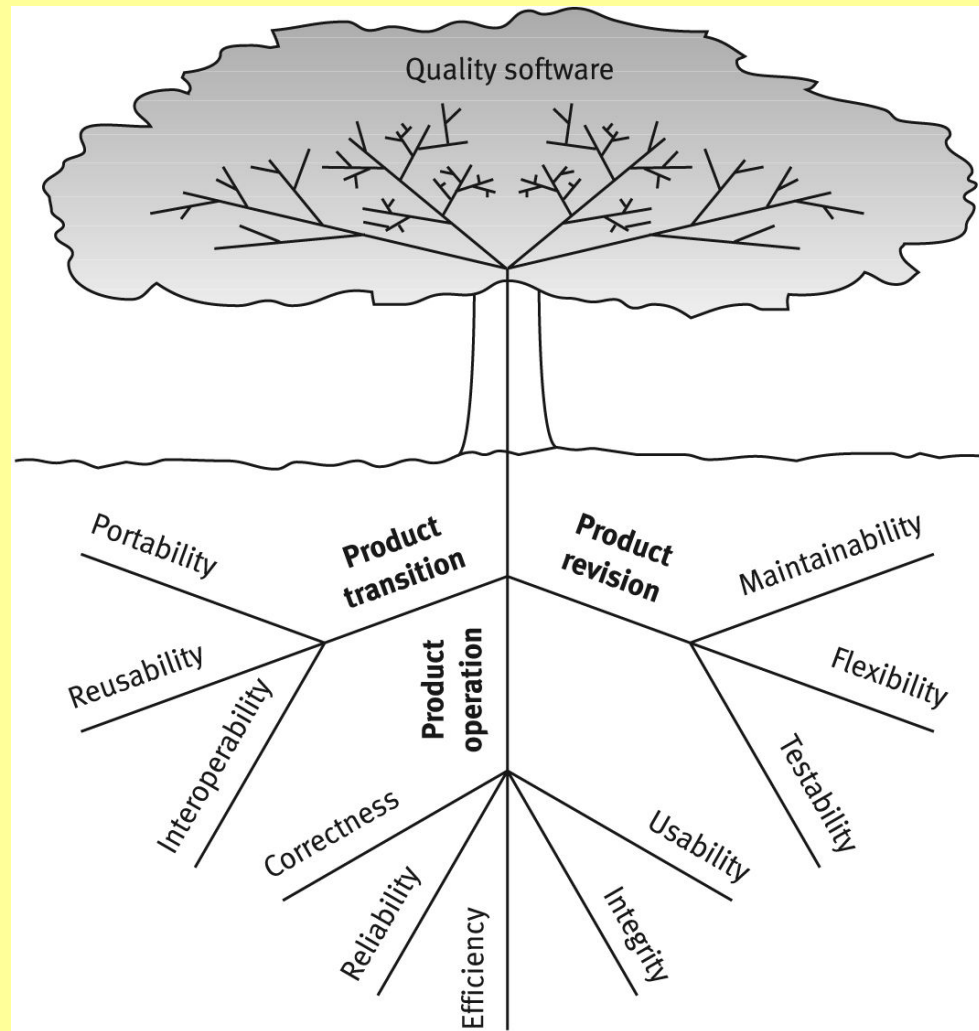
# McCall's software quality factors model



**Note:** not much apparent emphasis on functionality – only in correctness.



# McCalls factor model tree



# Product operation factors

- Correctness
- Reliability
- Efficiency
- Integrity
- Usability

How well does it run and ease of use.

# McCall's Quality Factors

## Category: Product Operation Factors

- **1. Correctness.**
- **Please note that we are asserting that 'correctness' issues are arising from the requirements documentation and the specification of the outputs...**
- **Examples include:**
  - **Specifying accuracies for correct outputs** at, say, NLT <1% errors, that could be affected by inaccurate data or faulty calculations;
  - **Specifying the completeness of the outputs** provided, which can be impacted by incomplete data
  - **Specifying the timeliness of the output** (time between event and its consideration by the software system)
  - **Specifying the standards** for coding and documenting the software system

# McCall's Quality Factors

## Category: Product Operation Factors

- **2. Reliability Requirements.** (remember, this quality factor is specified in the specs!)
  - Reliability requirements deal with the failure to provide service.
    - Address failure rates either overall or to required functions.
  - Example specs:
    - A heart monitoring system must have a failure rate of less than one per million cases.
    - Downtime for a system will not be more than ten minutes per month (me)
- **3. Efficiency Requirements.** Deals with the hardware resources needed to perform the functions of the software.
  - Here we consider MIPS, MHz (cycles per second); data storage capabilities measured in MB or TB; communication lines (usually measured in KBPS, MBPS, or GBPS).
  - Example spec: simply very slow communications...

# McCall's Quality Factors

## Category: Product Operation Factors

- **4. Integrity** – deal with system security that prevent unauthorized persons access.
- **5. Usability Requirements** – deals with the scope of staff resources needed to train new employees and to operate the software system.
  - Deals with learnability, utility, and more. (me)
  - Example spec: A staff member should be able to process n transactions / unit time. (me)

# Product revision factors

- Maintainability
- Flexibility
- Testability

Can I fix it easily, retest, version it, and deploy it easily?

# McCall's Quality Factors

## Category: Product Revision Software Factors

- These deal with requirements that affect the complete range of software maintenance activities:
  - corrective maintenance,
  - adaptive maintenance, and
  - perfective maintenance
- **1. Maintainability Requirements**
  - The degree of effort needed to identify reasons (find the problem) for software failure and to correct failures and to verify the success of the corrections.
  - Deals with the modular structure of the software, internal program documentation, programmer manuals
  - Example specs: size of module  $\leq 30$  statements.

# McCall's Quality Factors

## Category: Product Revision Software Factors

- **2. Flexibility Requirements** – deals with resources to change (adopt) software to different types of customers that use the app perhaps a little differently;
  - May also involve a little perfective maintenance to perhaps do a little better due to the customer's perhaps slightly more robust environment.
- **3. Testability Requirements** –
  - Are intermediate results of computations predefined to assist testing?
  - Are log files created?
  - Does the software diagnose itself prior to and perhaps during operations?



# Product transition factors

- Portability
- Reusability
- Interoperability

Can I move the app to different hardware? Interface easily with different hardware / software systems; can I reuse major portions of the code with little modification to develop new apps?

# McCall's Quality Factors

## Category: Product Transition Software Quality Factors

- **1. Portability Requirements:** If the software must be ported to different environments (different hardware, operating systems, ...) and still maintain an existing environment, then portability is a must.
- **2. Reusability Requirements:** Are we able to reuse parts of the app for new applications?
  - Can save immense development costs due to errors found / tested.
  - Certainly higher quality software and development more quickly results.
  - Very big deal nowadays.

# McCall's Quality Factors

## Category: Product Transition Software Quality Factors

- **3. Interoperability Requirements:** Does the application need to interface with other existing systems
  - Frequently these will be known ahead of time and plans can be made to provide for this requirement during design time.
    - Sometimes these systems can be quite different; different platforms, different databases, and more
  - Also, industry or standard application structures in areas can be specified as requirements.

# Alternatives

- Some other SQA professionals have offered essentially renamed quality factors.
- One has offered 12 factors; another 15 factors.
- Totally five new factors were suggested
- Evans and Marciniak offer two ‘new’ ones:
  - Verifiability and Expandability
- Deutsch and Willis offer three ‘new’ ones.
  - Safety
  - Manageability, and
  - Survivability

# McCall's factor model and alternative models

| No. | Software quality factor | McCall's classic model | Alternative factor models |                          |
|-----|-------------------------|------------------------|---------------------------|--------------------------|
|     |                         |                        | Evans and Marciniak model | Deutsch and Willis model |
| 1   | Correctness             | +                      | +                         | +                        |
| 2   | Reliability             | +                      | +                         | +                        |
| 3   | Efficiency              | +                      | +                         | +                        |
| 4   | Integrity               | +                      | +                         | +                        |
| 5   | Usability               | +                      | +                         | +                        |
| 6   | Maintainability         | +                      | +                         | +                        |
| 7   | Flexibility             | +                      | +                         | +                        |
| 8   | Testability             | +                      |                           |                          |
| 9   | Portability             | +                      | +                         | +                        |
| 10  | Reusability             | +                      | +                         | +                        |
| 11  | Interoperability        | +                      | +                         | +                        |
| 12  | Verifiability           |                        | +                         | +                        |
| 13  | Expandability           |                        | +                         | +                        |
| 14  | Safety                  |                        |                           | +                        |
| 15  | Manageability           |                        |                           | +                        |
| 16  | Survivability           |                        |                           | +                        |

# Alternatives

Evans and Marciniak offer Verifiability and Expandability

- **1. Verifiability Requirements** addresses design and programming features that allow for efficient verification of design and programming;
  - This does not refer to outputs; rather, structure of code; design elements and their dependencies, coupling, cohesion; patterns...
- apply to modularity, simplicity, adherence to documentation and programming guidelines, etc.
- **2. Expandability Requirements** really refers to scalability and extensibility to provide more usability.
  - Essentially this is McCall's **flexibility**

# Alternatives

Deutsch and Willis offer Safety, Manageability, and Survivability

- **1. Safety Requirements** address conditions that could bring the equipment or application down especially for controlling software, as in setting alarms or sounding warnings.
  - Especially important to process control / real time software such as that running conveyor belts or instrumentation for ordinance...
- **2. Manageability Requirements** refer to tools primarily administrative to control versions, configurations and change management / tracking.
  - We must have tools to manage versions and various configurations that may vary from customer to customer.
- **3. Survivability Requirements** refer to MTBF, or continuity of service, as well as MTTR (mean time to recover).
  - Appears to be quite similar to Reliability in McCall's model

# Comparisons

- These all are pretty close.
- Both models do add **Verifiability**,
- I like this one; addresses design and programming.  
(programming conventions / standards, etc.)
  - As we have developed as a discipline, I think this is essential.
- **Safety** is clearly important as computers control more and more of what we do especially in both hardware and in software.
  - New cars will now sound an alarm as we back up; software will sound when power is interrupted. This is important.



# So, Who Cares?

- Both developers and clients need to care. One group may care more than the other for certain quality factors.
- Certainly the nature of the app dictates more concern on some of these factors than others (safety, for example)
- In the Rational Unified Process, we call this component of the requirements the SRS (software requirement specifications, which contains these ‘non-functional requirements’ which are requirements in every sense of the word – just not functional requirements, as the term is normally used.
- **What are your feelings about these quality factors?**

# Homework #3

You may work alone or with one student on this assignment:

- 1. Please address each of the McCall's quality factors and address how each apply to software and/or hardware (not using the examples in the chapter). Please add verifiability and safety.
  - Develop a short paper in Word that has no less than a single paragraph addressing each of these quality factors. (This may take a two pages)
  - Table 3.3 may provide some assistance.
- 2. Answer question 3.2
- 3. Question 3.5 discusses qualitative and quantitative requirements with the assertion that quantitative alternatives should be preferred. What are your thoughts on these. (no more than one page, please on #3)
- Due date and date for in-class discussion will be announced.

# Homework Chapter 3

- You are to answer in essay format answers to questions 3.4 and 3.6.
- You are to turn them in via Blackboard Assignment

# Team Discussion

- Team 3 is to lead the discussion on the following questions:
- 3.3
- 3.4
- 3.5